

DevOps Lab: Automated Jenkins Master-Slave Java WAR Deployment to Tomcat

Difficulty Level: Medium **Duration:** 30 Minutes

Scenario

Your company runs a legacy Java enterprise web application packaged as a Web Application Archive (.war) file. To protect the Jenkins control plane from high resource usage during compilation and to secure direct server deployments, you are tasked with designing and implementing a distributed Jenkins Master-Slave architecture.

The Jenkins Master will serve as the orchestration controller, listening for automatic **GitHub webhook triggers** on updates to the master branch. The actual compilation, packaging (using **Maven**), and deployment (using **Apache Tomcat**) will take place on a dedicated Jenkins Slave node. Communication between the Master and Slave must be secured using SSH keys. Once the Maven build completes, the pipeline must deploy the .war package to a Tomcat instance running on the Slave, verify the deployment, and generate an audit report.

Task Objectives

To complete the setup of this distributed architecture, you must perform the following actions: 1. **Configure Master-Slave SSH Connection:** Set up passwordless key-based SSH access between the Jenkins Master and Slave. 2. **Register the Jenkins Slave Node:** Add the Slave as a permanent agent in the Jenkins dashboard via the SSH launcher. 3. **Configure GitHub Webhook:** Integrate a webhook from GitHub to automatically trigger builds on Master branch push events. 4. **Build the Java Web App:** Create a pipeline or build job restricted to run exclusively on the Slave node that compiles and packages the app using Maven (`mvn clean package`). 5. **Deploy to Apache Tomcat:** Copy the packaged .war file to the local Tomcat webapps directory on the Slave and verify it is running. 6. **Generate Evaluation Logs:** Write the pipeline outcomes to a structured audit report file.

Requirements

1. VM Infrastructure & Ports

- **Jenkins Master Node (jenkins-master):**
 - Runs Jenkins Master.
 - Inbound ports allowed: Port 22 (SSH) from your IP, Port 8080 (HTTP) from Anywhere.
- **Jenkins Slave Node (jenkins-slave):**
 - Runs Apache Tomcat 9/10, Java JDK/JRE, and Maven.
 - Inbound ports allowed: Port 22 (SSH) from the Master's private IP, Port 8080 (Tomcat HTTP) from Anywhere.
 - Remote root directory: /home/ubuntu/jenkins-slave.

2. Connectivity & Credentials

- Set up an SSH key pair on the Master. Add the public key to the /home/ubuntu/.ssh/authorized_keys file on the Slave node.
- Configure a new credentials item in Jenkins of type **"SSH Username with private key"** (using username ubuntu and the private key generated on Master).
- Register the Slave in Jenkins Nodes under the name jenkins-slave, applying the label java-builder.

3. Pipeline Build & Deployment Steps

- Create a Freestyle or Pipeline project in Jenkins named Tomcat-Deployment-Eval.
- Configure the job to run exclusively on node label java-builder.
- Trigger the job automatically using **GitHub hook trigger for GITScm polling**.
- In the build steps:
- Clone the repository.
- Run the Maven packaging command: mvn clean package.
- Deploy the resulting WAR file by copying it to the local Tomcat webapps directory: bash cp target/*.war /var/lib/tomcat9/webapps/app.war
- Generate an evaluation audit log at /home/ubuntu/build-logs/tomcat-deploy.log on the Slave containing the build status.

Expected Workflow



Sample Evaluation Report

Your pipeline should write an audit log to /home/ubuntu/build-logs/tomcat-deploy.log on the Slave matching this format:

```
GIT_CLONE=SUCCESS
MAVEN_BUILD=SUCCESS
TOMCAT_DEPLOYMENT=SUCCESS
PIPELINE_NAME=Tomcat-Deployment-Eval
BUILD_NUMBER=15
FINAL_STATUS=SUCCESS
TIMESTAMP=Fri Jun 19 04:30:00 UTC 2026
```

Deliverables

The final implementation must consist of:

Deliverable	Description
Jenkins Master Setup	Active Master EC2 instance serving Jenkins on HTTP port 8080.
Active SSH Agent Node	Connected permanent node jenkins-slave showing as "Online" in Jenkins.
Tomcat Web Server	Active Tomcat service listening on port 8080 on the Slave.
GitHub Webhook Integration	Configured repository webhook that automatically triggers Jenkins on push events.
Maven Deployment Job	Project named Tomcat-Deployment-Eval restricted to label java-builder.
Evaluation Audit Report	A structured status log at /home/ubuntu/build-logs/tomcat-deploy.log on the Slave.

Technology Stack

Technology	Purpose
Amazon EC2	Compute infrastructure for Master and Slave hosts.
Ubuntu Linux	Server operating systems.
Jenkins	CI/CD Server orchestrating builds.
Maven	Build tool used to compile code and generate .war packages.
Apache Tomcat	Web Application Server hosting the packaged Java web app.
SSH	Secure channel for Agent connectivity and artifact deployment.

Verification & Grading Criteria

Your configuration will be automatically graded based on the following verification checks:

Test Case	Requirement	Validation Method	Marks
TC1	Jenkins Master-Slave Connection	Verifies that node <code>jenkins-slave</code> is configured in Jenkins, connected via SSH, and is online.	4 Marks
TC2	GitHub Webhook Trigger	Verifies that a GitHub webhook is active and auto-triggers the Jenkins job on code commits.	4 Marks
TC3	Maven Build Execution	Verifies that the Java code compiles successfully on the Slave and outputs the target <code>.war</code> package.	4 Marks
TC4	Tomcat Deploy Validation	Verifies that the <code>.war</code> application is successfully deployed to Tomcat's <code>webapps</code> directory on the Slave.	4 Marks
TC5	Pipeline Log & Report Generation	Verifies that the log file exists at <code>/home/ubuntu/build-logs/tomcat-deploy.log</code> on the Slave with success properties.	4 Marks

Total Score: 20 Marks

Optional Enhancements

For advanced practice, you can attempt to implement:

- Setup a Jenkinsfile pipeline script checked into GitHub.
- Automate Tomcat deployment using Tomcat Manager REST API instead of local copying.
- Secure HTTP web traffic using self-signed SSL/TLS certificates on Tomcat.
- Containerize the Tomcat deployment using Docker containers on the Slave.

Real-World Use Case

This Master-Slave configuration replicates enterprise-grade CI/CD architecture where compilation, unit testing, and deployments are segregated to separate agent nodes. This keeps the central orchestrator responsive, allows scale out of build infrastructure dynamically based on job queues, and isolates deployment permissions to secure network boundaries.